System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models

Sam Adè Jacobs
Microsoft Inc
Redmond, USA
samjacobs@microsoft.com

Masahiro Tanaka
Microsoft Inc
Redmond, USA
mtanaka@microsoft.com

Chengming Zhang
Microsoft Inc
Redmond, USA
v-chengmingz@microsoft.com

Minjia Zhang
Microsoft Inc
Redmond, USA
minjiaz@microsoft.com

Reza Yazdani Aminabadi
Microsoft Inc
Redmond, USA
rezayazda@microsoft.com

Shuaiwen Leon Song
Microsoft Inc
Redmond, USA
leonsong@microsoft.com

Samyam Rajbhandari
Microsoft Inc
Redmond, USA
samyamr@microsoft.com

Yuxiong He
Microsoft Inc
Redmond, USA
yhe@microsoft.com

Overview

- Computation in a typical transformer-based large language model (LLM) can be characterized by batch size, hidden dimension, number of layers, and sequence length (#tokens, i.e., words in a single input/output).
- Widely studied forms of parallelism are not targeted or optimized for long sequence transformer models.
- Existing works in sequence parallelism are constrained by memory and communication inefficiency, limiting their scalability to long sequence large models.
- We introduce Ulysses, a novel, portable, and effective methodology for enabling highly efficient and scalable LLM training with extremely long sequence length.

Overview

- Ulysses partitions input data along the sequence dimension and employs an efficient all-to-all collective communication for attention computation.
- Theoretical communication analysis shows that Ulysses maintains constant communication volume when sequence length and compute devices are increased proportionally (think weak scaling).
- Ulysses scales to more than 1 million context length and trains 2.5x faster with 4x longer sequence length than the existing methods.

Overview

Importance of efficiently supporting long sequence

- For example, multimodal foundation models such as ones that process speech, images and waveforms concurrently require long context reasoning over high dimensional inputs with long sequences [5, 20, 29].
- Long sequence length is equally critical for AI for science opening doors for better understanding of structure biology, health care, climate and weather forecasting [19] and large molecular simulation [36].
- For example, by adapting large language models with gene sequences, we can create language models that can learn the evolutionary patterns of genomes using simple alphabets and extremely long sequences (the human genome has 6.4 billion letters) [36].
- In health care, diagnostic predictive model conditioned on entire patient care record requires context of long sequences [9, 15].

Overview

Importance of efficiently supporting long sequence

Long Sequence Applications in Scientific Machine Learning (SciML):

- GenSLM [36] is one of the first whole-genome scale foundation models that can be generalized to other prediction tasks.
- GenSLMs utilization of genome-scale language models that learn the evolutionary landscape of SARS-CoV-2 genomes.
- GenSLMs can identify and classify new and emerging variants of pandemic-causing viruses, particularly SARS-CoV-2.
- To have a good understanding of the latent space, GenSLMs, and similar models require very long sequence support for both training and inference.

- ClimaX [19] is developed as a flexible and generalizable deeplearning model for weather and climate science.
- It can be trained using heterogeneous datasets that span different variables, spatiotemporal coverage, and physical groundings.
- The model extends the transformer architecture, incorporating novel encoding and aggregation blocks that enable the effective use of computing resources while maintaining utility.
- However, current architectures struggle to support inputs with global resolution beyond 3 km, unless model parallelism is employed.

Overview

Limitations of existing techniques

- Existing parallelism approach such as data, tensor and pipeline parallelism cannot address the scaling along sequence dimension.
- Existing sequence parallelism approaches are not effective because of memory-communication inefficiencies.

Overview

Contribution summary

- Sequence parallelism and its combination with model parameter sharding (e.g., ZeRO [25, 26]) for efficient training of long sequence LLMs.
- Ulysses partitions individual samples along the sequence dimension among participating GPUs.
- Right before the attention computation, it employs all-to-all communication collective on the partitioned queries, keys, and values such that each GPU receives the full sequence but only for a non-overlapping subset of the attention heads.
- This allows the participating GPUs to compute attention for different attention heads in parallel.
- Finally, Ulysses employs another all-to-all to gather the results along the attention heads while repartitioning along the sequence dimension.

Overview

Contribution summary

- Ulysses trains transformer models 4x larger sequence lengths than existing systems, while enabling training with sequences with more than 1 million tokens.
- Communication reduction of over 10x compared to existing systems, resulting in throughput improvements of up to 2.5x, and sustained throughput of over 175 TFlops/GPU (over 54% of hardware peak)
- Ulysses supports dense as well as sparse attention, and it works with efficient attention implementations such as FlashAttention [5].
- https://github.com/microsoft/DeepSpeed
- https://github.com/microsoft/Megatron-DeepSpeed

Background

Transformer architecture:

- The QKV tensors are fed to the attention block.
- Outputs of attention are inputs to the multilayer perceptron (MLP) or position-wise feed-forward block of transformer architecture.
- The attention block followed by MLP block are replicated multiple times to form an encoder, a decoder or an encoder-decoder transformer network.

- Data parallelism is limited when model is large and model parameter replication across devices is practically infeasible.
- ZeRO [25, 26] optimization addresses this problem by partitioning model parameters across available compute devices. Both data parallelism and ZeRO do not address the problem of large intermediate activations associated with long sequences. Moreover, large batches are known to have negative impacts on model quality [11].
- Our proposed approach is orthogonal to both data parallelism and ZeRO, meaning it can be used in conjunction with both methods. (How about tensor and pipeline parallelism?)
- Using sequence parallelism, we can maintain a fixed or reasonable global batch size on large systems, effectively mitigating the impact of large batch sizes on model convergence.

Background

Sequence parallelism serves two purposes:

- First, it can accelerate the time to solution for a fixed sequence length. In other words, sequence parallelism reduces the iteration time in proportion to the additional compute resources.
- Given more GPUs, the fixed global batch size is maintained and additional GPU resources are used for sequence parallelism. This also allows for training scenarios where the batch size per GPU (also known as microbatch size) is less than 1.

Background

Sequence parallelism serves two purposes:

- Second, sequence parallelism enables long sequence training or continual pretraining where the training context length gradually increases over time.
- Consider a real-world scenario of large-scale training on 1024 GPUs, microbatch size of 1 per GPU.
- To improve the quality of the pretrained model, a simple change in sequence length from 8K to 32K is required, which would result in a global batch size of approximately 32 million.
- However, increasing the global batch size (number of samples) is not an option due to its negative impact on model quality.
- Here, Sequence parallelism comes in handy as a system optimization technique that does not require laborious hyperparameter search.
- In this scenario, sequence parallelism allows 32K tokens to be split across multiple GPUs along the sequence dimension (e.g., a sequence parallel group of 4) without increasing the global batch size.

Background

- Similarly to our work, all existing works in sequence parallelism partition the input data along sequence dimension but differ in what input projections are partitioned and how partitions are aggregated and communicated for attention computation.
- Table 1 summarizes how Ulysses differs from other existing methods. Ulysses has communication efficiency advantage over the other two methods. It also benefits from extending ZeRO [25, 26] optimization for model parameter partitioning across both sequence and data parallel groups.

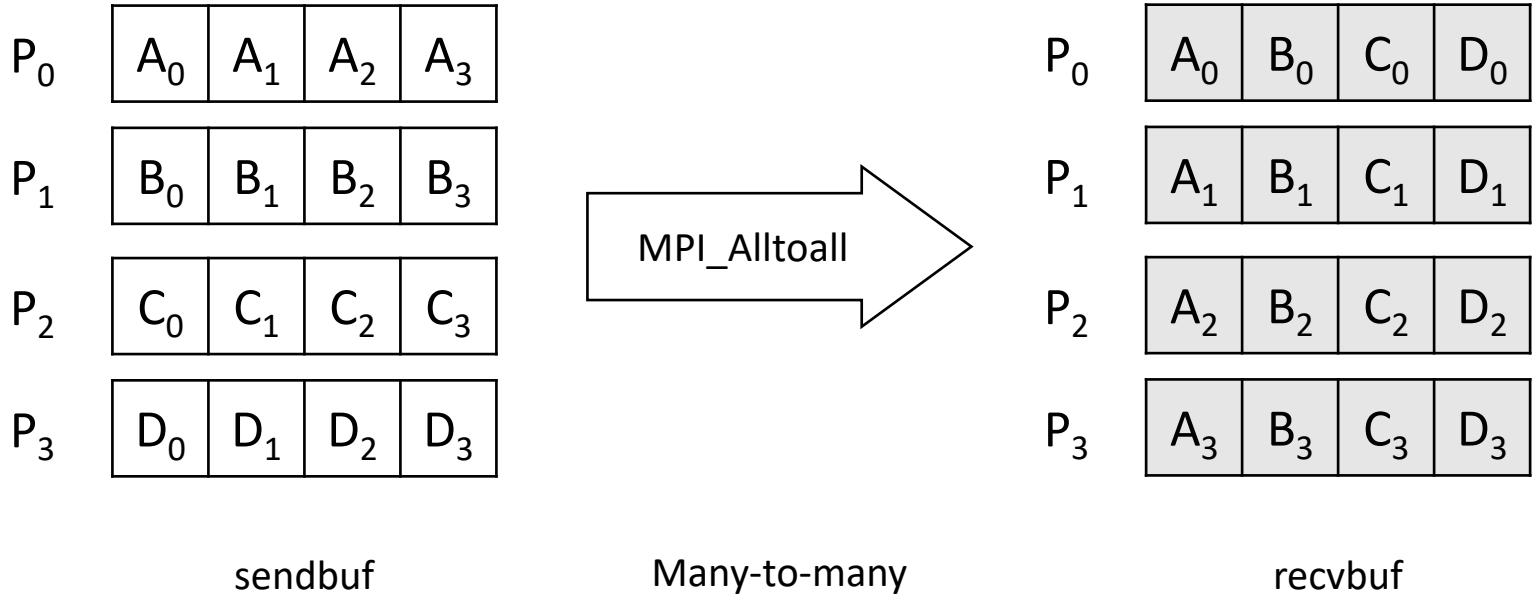
Table 1: Comparison of our work (Ulysses) to other sequence parallelism methods.

Methods	Comm complexity	Activation memory efficiency	Parameter memory efficiency	Attention agnostic	Ease of use
ColAI-SP [14]	$O(M)$	✓	✗	✗	✗
Megatron-SP [12]	$O(M)$	✓	✓	✓	✗
Ulysses	$O(M/P)$	✓	✓	✓	✓

Collective communication: Collective data movement - Alltoall

int **MPI_Alltoall**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf,
int recvcount, MPI_Datatype recvtype, MPI_Comm comm)

- All ranks to all processes of the communicator group



Proposed solution

- In a transformer, input sequences of size *N* is partitioned across *P* available devices.
- Each local *N/P* partition is projected into queries (*Q*), keys (*K*) and values (*V*) embeddings.
- Next, (*QKV*) embeddings are gathered into global *QKV* through highly optimized all-to-all collectives between the participating compute devices.
- Sequel to all-to-all collective is the attention computation per head in the form:

$$Outputcontext = Softmax((QK^T)/\sqrt{(d)})V$$

- After the attention computation, another all-to-all collective transforms output context tensor of attention computation to sequence (*N/P*) parallel for subsequent operators (MLP MatMul, layer norm etc) in the remaining modules of transformer layer block.
- Essentially, the computation pattern of Ulysses transitions from sequence parallelism to head parallelism, and then back to sequence parallelism. (Note the red arrows in the figure on the next slide.)
- Efficient all-to-all communication (using MPI, NCCL, RCCL etc.) is crucial for transitioning between these two modes of parallelism.

Proposed solution

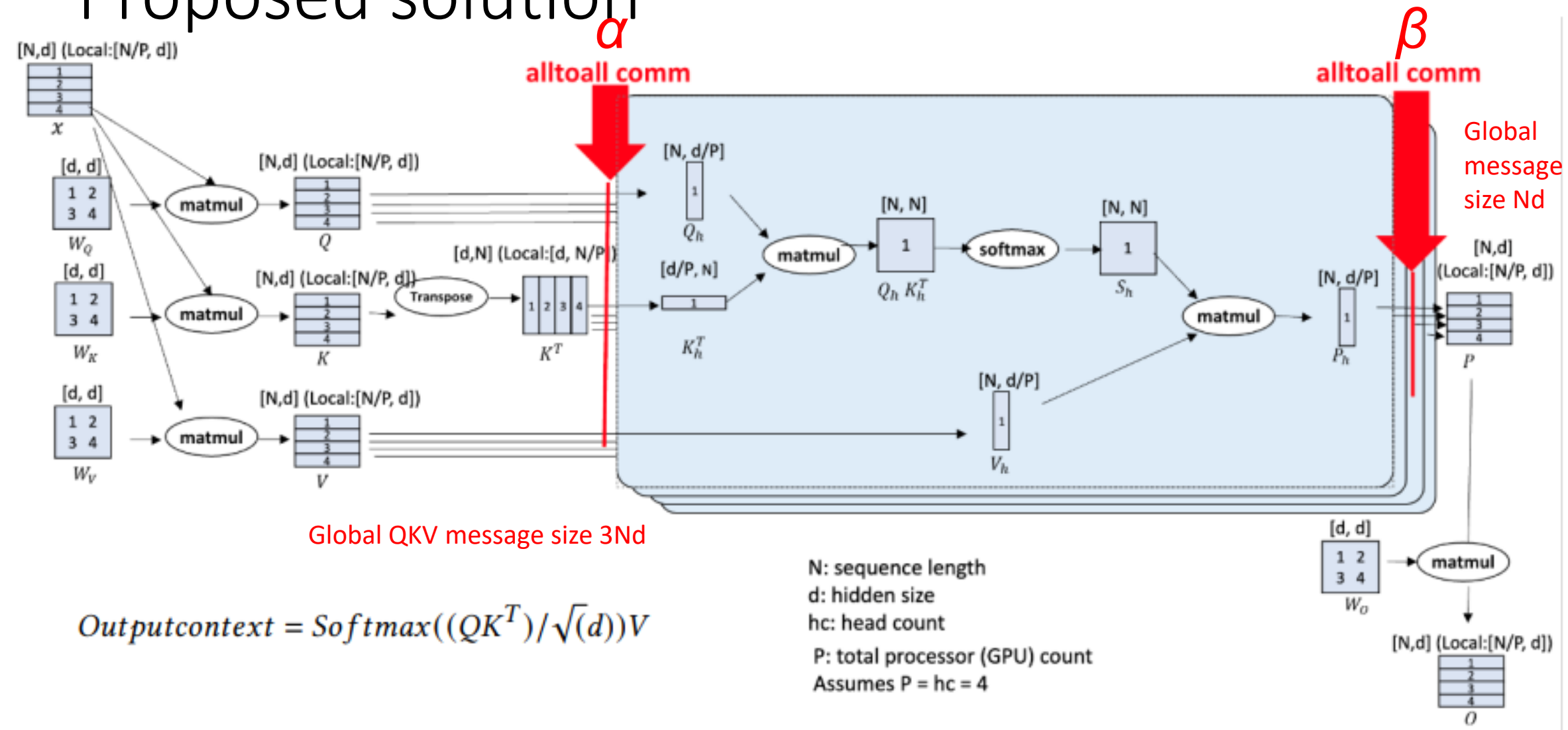


Figure 2: Ulysses core design

Proposed solution

Communication analysis

- What distinguishes Ulysses from the other existing long-sequence approaches is our much smaller aggregate communication volume and overall better scalability with increasing degree of sequence parallelism compared to existing solutions.
- The communication volume transmitted per link for an all-to-all for aggregate message of size *M* over *P* GPUs is *M/P*.
- For a transformer model with hidden size *d*, sequence length of *N*, and parallelism degree of *P*, Ulysses performs all-to-all for the *QKV* projections with an aggregate message size of *3Nd* before the attention computation (α in the figure on the previous slide).
- Another all-to-all for output context projection with a size *Nd* for each transformer layer (β in the figure on the previous slide).
- Therefore, Ulysses incurs an aggregate communication volume per link of *4Nd/P* (or with the complexity of *O(N/P)).*
- Communication volume is constant when both *N* and *P* are increased proportionally.

Proposed solution

Communication analysis

Limitations of existing techniques (the math is also available in the Megatron-LM paper):

- In existing approaches such as the Megatron-LM [12] and ColAI [14], sequence parallelism incur a communication volume that increases linearly with *N* regardless of *P*, resulting in the communication complexity of *O(N)*.
- Megatron-LM conducts an all-gather prior to computing *QKV* and an reduce-scatter after attention computation.
- It needs additional all-gather and reduce-scatter for an MLP block.
- The cost of each all-gather and reduce-scatter remains constant at size *N*, regardless of *P*. (There is a typo in the paper. It should be *N*, not *M.*)
- As a result, the communication cost for Megatron-LM's sequence parallelism is *4Nd* for each transformer layer, which is *P* times larger than that of Ulysses (one transformer layer per device, total *P* devices).

Table 2: Ulysses vs Megatron-LM communication collective benchmark on 8 A100 GPUs

Methods	Sequence Length	Hidden Size	Reduce Scatter	All Gather	AllTo All	Total Comm Time
Megatron-LM	256K	4K	4GB	4GB	NA	34ms
Ulysses	256K	4K	NA	NA	8GB	4.9ms

The reason for the faster data transfer in all-to-all is that each GPU pair involved only has to send a fraction, $1/P$, of the total amount of data, without contention (see Figure 3 on the next slide).

Proposed solution

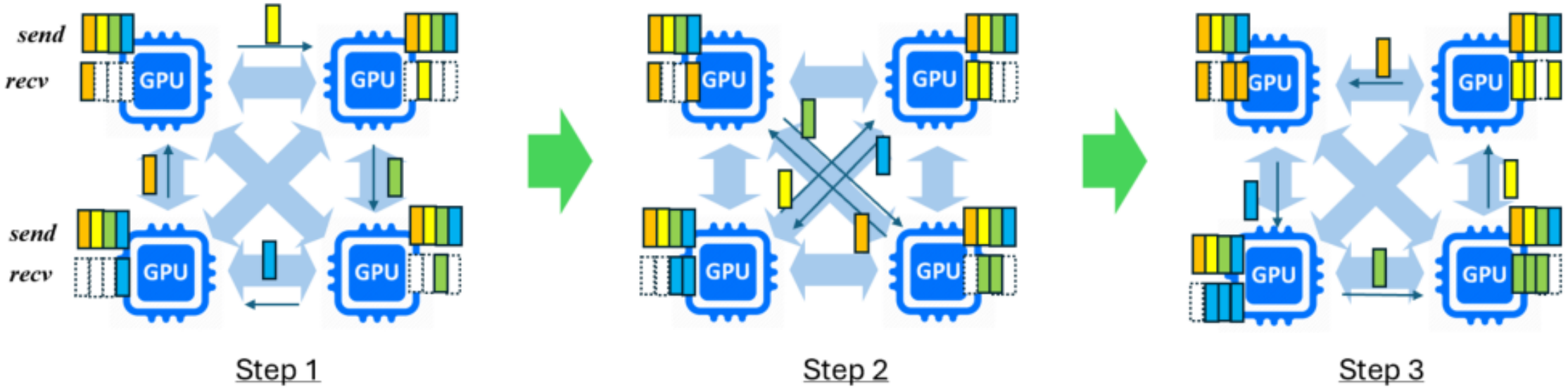


Figure 3: All-to-all collective communication illustration for our method. Each GPU (rank) sends $1/P$ of total volume at each step without contention. States of receive buffer while messages are in transmission are shown in dotted lines.

Proposed solution

Implementation of Ulysses in ZeRO Redundancy Optimizer

- ZeRO (ZeRO Stage 3 or ZeRO-3 in particular) optimizes memory usage by partitioning model states across data parallel ranks (comparable to tenson parallelism).
- With sequence parallelism, training data can be considered in both batch (sample) and sequence dimensions and the associated parallel groups combined to form a larger group for ZeRO parallelism.
- We extend ZeRO partitioning to combination of data parallel and sequence parallel ranks.
- In Ulysses, ZeRO partitions model states across both sequence and data parallel group and collects per-rank partitions (all-gather) when they are needed.
- Similarly, gradients are reduced across both data and sequence parallel ranks for parameter update.

Proposed solution

Implementation of Ulysses in ZeRO Redundancy Optimizer

- The general property of Ulysses stems from the modular nature of its core design: an attention-centric sequence parallelism design.
- Ulysses' implementation of distributed attention module is general enough to support any attention: e.g., self-attention, cross-attention, causal attention in both their dense and sparse counterparts, and their various optimized kernels that support long-sequence at local attention level such as different versions of FlashAttention.
- Prior to attention computation is sequence parallelism of N/P partition, attention computation is head parallelism with full attention per head but just with fewer heads, thus attention computation can be replaced with any type of attention mechanisms, e.g., dense attention and various forms of sparse attention.

Evaluation

- The testbed is the ThetaGPU Cluster at the Argonne National Laboratory.
- This cluster is built with NVIDIA DGX A100 nodes.
- Each node is equipped with 2 AMD Rome CPUs and 8 NVIDIA A100 GPUs, each with 40GB of memory. These nodes utilize high-performance NVLink and NVSwitch technologies for intra-node communication.
- The interconnect system features eight ports of HDR200 InfiniBand, delivering an impressive aggregated bandwidth of 200GB/s.

Evaluation

Methodology

- We employed the standard GPT architecture [24], but with varying sizes.
- For instance, the GPT-7B model has a hidden dimension of 4096, 32 heads, and 32 layers.
- The GPT-30B model has a hidden dimension of 6144, 64 heads, and 64 layers.

Evaluation

Sequence length scalability

- Strong scaling of sequence length up to 1 million tokens on the 1.2 billion-parameter GPT model (GPT- 1.2B).
- Ulysses allows increasing sequence length linearly with the number of GPUs and sequence length scales linearly relative to and maintains similar computation throughput across different sequence lengths at appropriate GPU count.
- Where is speedup?

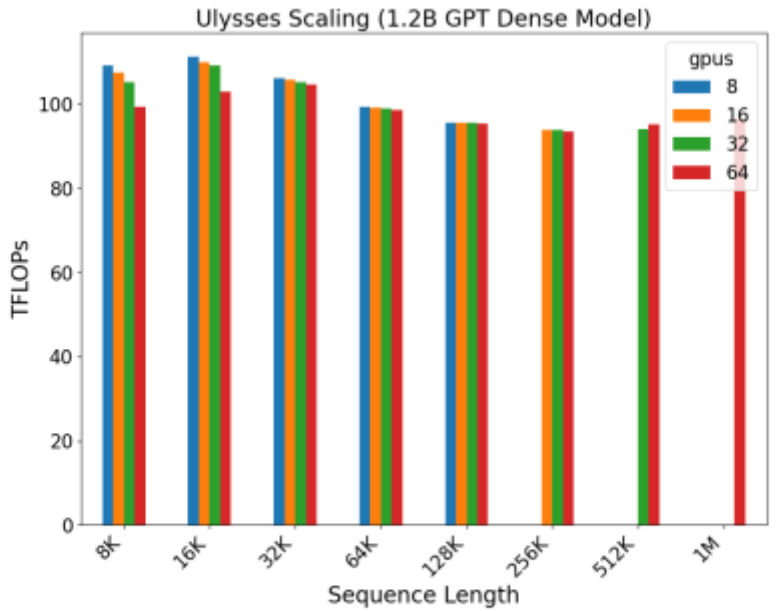


Figure 4: Ulysses strong scalability evaluation of GPT-1.2B at different sequence length and GPU counts

Evaluation

Dense attention evaluation

- We evaluated Ulysses on 7 billion and 30 billion parameter GPT dense attention models and compared against Megatron-LM's sequence parallelism (Megatron LM) and Colosal AI sequence parallelism (ColAI-SP) on 32 and 64 A100 GPUs, respectively.
- Running various sequence lengths. We chose the sequence parallelism degree and micro-batch size that produced the best performance (measured as TFLOPs) for the three methods, this we call optimal (batch size-sequence length) configurations.
- Figures 5 and 6 show that Ulysses consistently outperforms Megatron-LM and ColAI-SP for the sequence length that can be run with them. In addition, Ulysses can run longer sequence than the two existing methods.

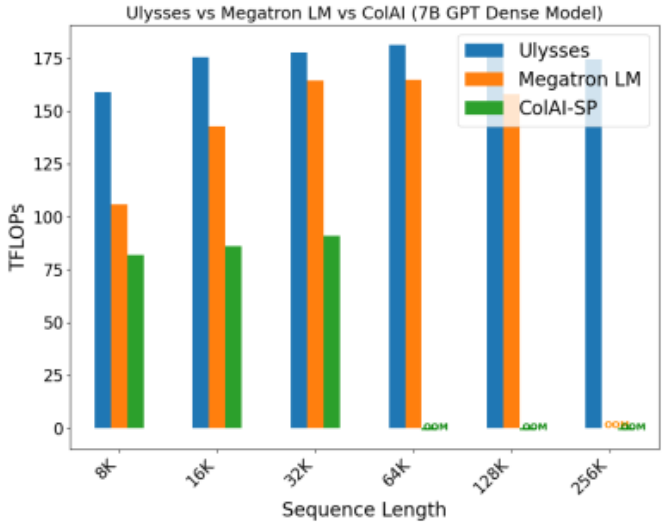


Figure 5: Evaluation of Ulysses vs Megatron LM vs ColAI-SP on GPT-7B parameter model with dense attention (32 GPUs)

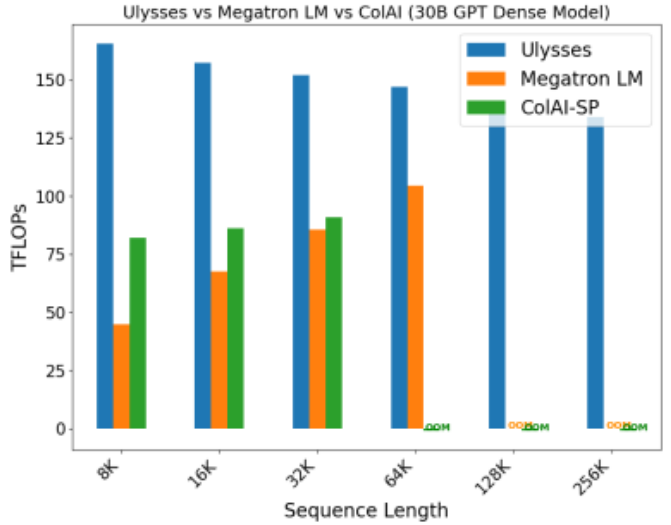


Figure 6: Evaluation of Ulysses vs Megatron LM vs ColAI-SP on GPT-30B parameter model with dense attention (64 GPUs)

Evaluation

Dense attention evaluation

Observations:

- Ulysses in combination with ZeRO-3 parameter sharding across both data and sequence parallel groups fits more samples than Megatron-LM and ColAI-SP because of the memory optimization leading to higher throughput.
- Ulysses benefits from efficient all-to-all communication relative to all-gather reduce-scatter and ring-style P2P communication as applied in Megatron-LM and ColAI-SP sequence parallelism.
- For dense attention at long sequence length, the throughput is primarily determined by local attention computation due to quadratic computation complexity of attention; therefore, performance gap between Ulysses and the two existing methods closes for sequence length that can be run with them.

Evaluation

Sparse attention evaluation

- Similarly, we evaluated Ulysses on sparse attention models.
- We observed similar trends with sparse attention as in dense attention experiments.

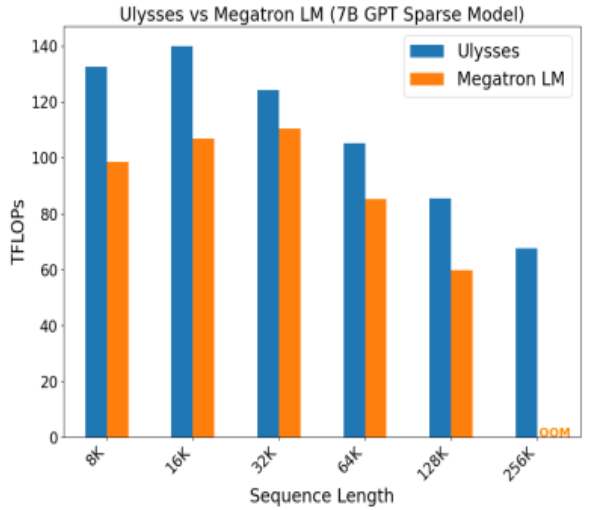


Figure 7: Evaluation of Ulysses and Megatron LM on GPT-7B parameter model with block sparse attention (32 GPUs)

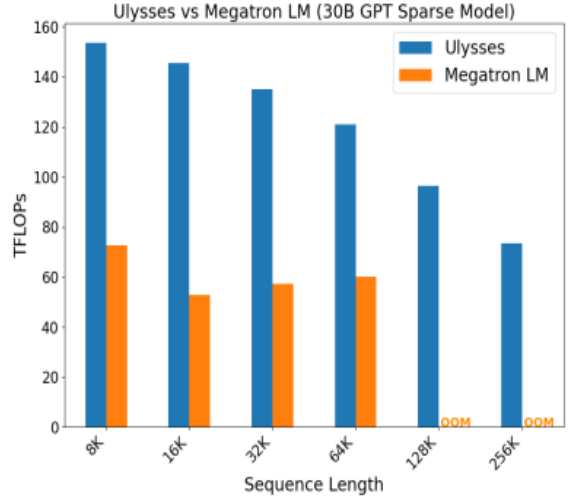


Figure 8: Evaluation of Ulysses and Megatron LM on GPT-30B parameter model with block sparse attention (64 GPUs)

Evaluation

Sparse attention evaluation

Observations:

- Recall communication complexity of Ulysses as *O(M/P)* compared to *O(M)* of Megatron-LM, where *M* is message size and *P* is number of GPUs.
- Ulyssess performs all-to-all after computing QKV resulting in 3 more volume than Megatron. With *P* set to 32 for GPT-7B model, 3*M*/32 is approximately 10x in communication reduction compared to Megatron-LM.
- Currently Ulysses' throughput is bottlenecked by the local sparse attention implementation, and as a result Ulysses throughput decreases as the sequence length increases.
- A noteworthy observation is that the decreasing performance gap between Ulysses and Megatron-LM observed in dense attention evaluation is less pronounced in sparse attention evaluation, because the attention computation in sparse attention is less dominant compared to dense attention.

Evaluation

Ulysses and ZeRO: a comparative analysis of sequence length capabilities

- ZeRO-3 partitions model weights, while Ulysses partitions sequences.
- Although ZeRO-3 enables training with large models, it has a significant limitation when training with long sequences due to the large activations associated with these sequences.
- Table 3 - training with the GPT-7B and GPT-30B dense models, maximum sequence lengths achievable.

Table 3: ZeRO and Ulysses: A comparative analysis of sequence length capabilities

Methods	Model Size	GPUs	Max Seqlen	TFLOPs at Max Seqlen
Ulysses	7B	32	256K	174.49
ZeRO	7B	32	16K	175.3
Ulysses	30B	64	256K	134.03
ZeRO	30B	64	8K	165.52

Evaluation

Ulysses and ZeRO: a comparative analysis of sequence length capabilities

- Table 4 - sequence length is fixed to 131,072 tokens.
- Table 5 - we increase the GPU count proportionally to the increase in the sequence length (think weak scaling).
- For both evaluations, we used the GPT-7B dense model with a global batch size of 8.
- Attention computation, a function of sequence length, is quadratic in complexity.
- In other words, as we increase the sequence length, the work increases quadratically. This is why time is increasing in Table 5, weak scaling-like experiment.

Table 4: Ulysses parallel scaling study with fixed sequence length

Seqlen	GPUs	Time (ms)	TFLOPs
128K	64	32432.1333	165.526
128K	128	17052.5143	157.41
128K	256	9886.7	136.09

Table 5: Ulysses parallel scaling study with varying sequence length

Seqlen	GPUs	Time (ms)	TFLOPs
64K	64	9676.76	161.362
128K	128	17052.5143	157.41
256K	256	33486.5	147.4

Evaluation

Convergence study

- Figure 9 shows the convergence of a 1.3 billion GPT model with 32K sequence length on 8 A100 GPUs with the degree of sequence parallelism set to 4 for both the Ulysses and Megatron-LM sequence parallelism. For Ulysses, we evaluated the convergence with different ZeRO stages.

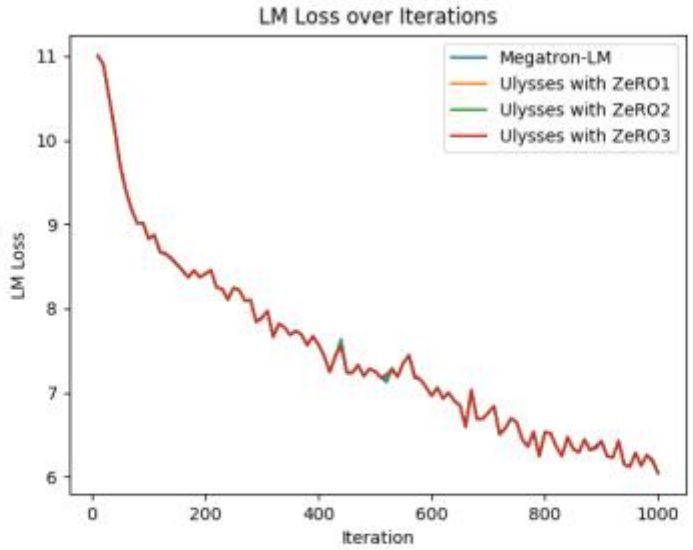


Figure 9: Convergence evaluation of Ulysses with different ZeRO memory optimization stages

Evaluation

Case study: training real-world science models with Ulysses

- We use an open source climate model, named *ClimaX* [19] (one of the first foundation models for weather and climate modeling tasks), to showcase long-sequence training for higher-resolution images in time series.
- ClimaX employs Vision Transformers (ViT) [8] and treats forecasting image modeling as a sequence-to-sequence problem.
- From the aspect of climate science, achieving highly accurate and localized weather forecasts necessitates one to resolve atmospheric dynamics at sub-kilometer spatial scales.
- To achieve such detailed forecasts across the globe, weather foundation models need to process extremely high-resolution images with dozens of channels, thereby leading to inputs with very large sequence lengths.
- For example, a target is to support inputs with a global resolution greater than 3 km, corresponding to the image size of 6000 × 12000.

Evaluation

Case study: training real-world science models with Ulysses

Challenges:

- Increasing the resolution of image inputs quadratically increases the image token sequence length. For instance, the image sequence length of a higher-resolution 4096 × 4096 image can result in a sequence with $64K$ image tokens.
- Given that memory complexity with respect to the number of tokens in the self-attention layers in ViT models also increases quadratically, higher-resolution images can easily cause out-of-memory (OOM) issues.

Evaluation

Case study: training real-world science models with Ulysses

- To address this challenge (i.e., OOM), we have integrated Ulysses into ClimaX model training, which is built on top of PyTorch Lightning [22].
- Figure 10 shows the maximum sequence length comparisons in the number of image tokens between Ulysses and the PyTorch Lightning baseline (with ZeRO stage 3 enabled) with the increasing number of GPUs.
- The original ClimaX model could only train 768 × 768 × 48 images (equivalent to the sequence length of 108K image tokens) with a batch size of 1 on 64 NVIDIA A100 GPUs. Further increasing the image resolution resulted in an out-of-memory (OOM) issue.
- With Ulysses, we managed to reduce ClimaX's peak memory consumption under the same image resolution. More importantly, thanks to Ulysses' critical memory saving techniques, we are now able to train ClimaX with even higher resolutions, up to 1536 × 1536 × 48 (equivalent to sequence length of 432K image tokens), 4× longer than the baseline implementation.

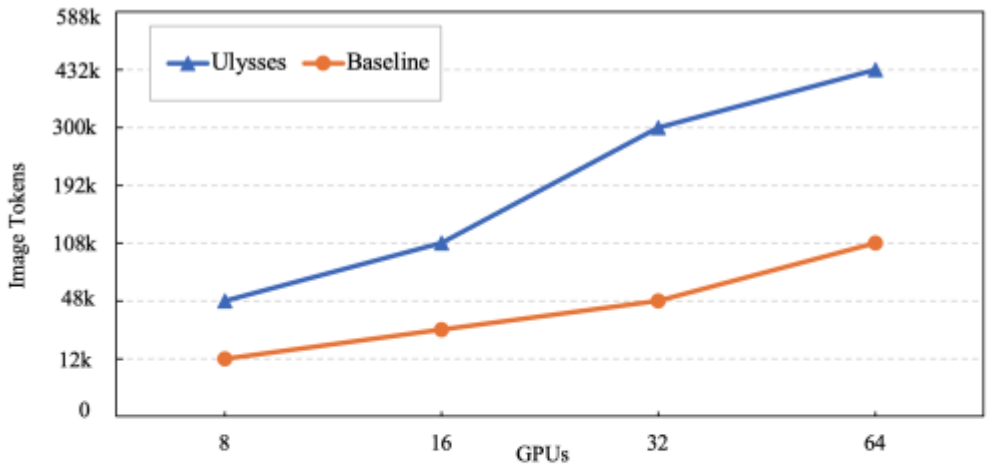


Figure 10: Evaluation of Ulysses and Baseline on the ClimaX model. The corresponding image sizes of image tokens of 12k, 48k, 108k, 192k, 300k, 432k, 588k are 256×256×48, 512×512×48, 768 × 768 × 48, 1024 × 1024 × 48, 1280 × 1280 × 48, 1536 × 1536 × 48, 1792 × 1792 × 48, **respectively.**